

```
In [1]: a = 5  
print(type(a))
```

```
<class 'int'>
```

```
In [2]: a1 = (5,6,7)  
print(type(a1))
```

```
<class 'tuple'>
```

```
In [4]: class laptop():  
        def config():  
            print('i7', 16)  
lap = laptop()
```

```
In [6]: class laptop():  
        def config():  
            print('i7', 16)  
lap = laptop()  
laptop.config()
```

```
i7 16
```

```
In [7]: class laptop():  
        def config():  
            print('i5', 16)  
lap = laptop()  
laptop.config()
```

```
i5 16
```

```
class laptop():  
    def config():  
        print('i7', 16)  
lap = laptop()  
lap.config()
```

```
In [9]: class laptop():  
        def config(self):  
            print('i7', 16)  
  
lap = laptop()  
lap.config()
```

i7 16

```
In [10]: class laptop():
          def config():
              print('i7', 16)

          lap = laptop()
          laptop.config()
```

i7 16

```
In [11]: class laptop():
          def config(self):
              print('i7', 16)

          lap = laptop()
          lap1 = laptop()

          lap.config()
          lap1.config()
```

i7 16

i7 16

```
In [13]: class computer():
          def config():
              print('i5', '8gb')

          a = 8.8
          print(type(a))

          com1 = computer()

          print(type(com1))
```

<class 'float'>

<class '__main__.computer'>

```
In [14]: class laptop():
          def config(self):
              print('i7', 16)
```

```
lap = laptop()  
  
laptop.config(lap)
```

i7 16

```
In [15]: class laptop():  
         def config(self):  
             print('i7', 16)  
  
         lap = laptop()  
         lap1 = laptop()  
         lap3 = laptop()  
  
         lap.config()  
         lap1.config()
```

i7 16

i7 16

Special method

```
In [16]: class computer():  
         def config(self):  
             print('i5', '8gb')  
  
         com1 = computer()
```

```
In [17]: class computer:  
         def __init__(self):  
             print('i5', '8gb')  
  
         com1 = computer()
```

i5 8gb

```
In [18]: class computer:  
         def __init__(self):
```

```
print('i5', '8gb')

com1 = computer()
com2 = computer()
com3 = computer()
```

i5 8gb
i5 8gb
i5 8gb

```
In [19]: class computer():
          def __init__(self):
              print('i5', '8gb')

          com1 = computer()
          com1.__init__()
```

i5 8gb
i5 8gb

```
In [20]: class computer():
          def __init__(self):
              print('i5', '8gb')

          com1 = computer()
          com2 = computer()

          com1.__init__()
          com2.__init__()
```

i5 8gb
i5 8gb
i5 8gb
i5 8gb

```
In [21]: class computer():
          def config(self):
              print('i5', '8gb')

          class computer():
              def __init__(self):
                  print('i5', '8gb')
```

```
com1 = computer()
```

i5 8gb

```
In [22]: class computer():
          def __init__(self):
              print('i3', 6)

          def config(self):
              print('i5', '8gb')

          com1 = computer()
          com1.config()
```

i3 6

i5 8gb

```
In [23]: class computer():
          def __init__(self):
              print('i3', 6)

          def config(self):
              print('i5', '8gb')

          com1 = computer()
          com1.config()
```

i3 6

i5 8gb

```
In [24]: class computer():

          def config(self):
              print('i5', '8gb')
          def config2(self):
              print('i7', '32gb')
          def __init__(self):
              print('in init')

          com1 = computer()
          com2 = computer()
```

```
com1.config()
com1.config2()
```

```
in init
in init
i5 8gb
i7 32gb
```

```
class computer():

    def __init__(self):
        print('in init')

    def config(self):
        print('i5', '8gb')
```

```
com1 = computer('i5', 8)
```

```
com1.config()
```

```
class computer:

    def __init__(self): #__init__ means to initialize the variable
        print('in init')

    def config(self):
        print('i5', '8gb')
```

```
com1 = computer('i5', 8) # com1 has cpu i5 & ram is 8 com2 = computer('i7', 16) # com2 has cpu intel 5 & ram is 8 as we have 2 computer we
have different ram & cpt
```

```
com1.config() com2.config()
```

```
In [27]: class computer():
```

```
    def __init__(self, cpu, ram, price): # cpu & ram as argument
```

```

        print('in init')

    def config(self):
        print('i5', '8gb', 50000)
        print('i7', 16, 100000)

com1 = computer('i5', 8, 50000)
com2 = computer('i7', 16, 100000)

com1.config()
com2.config()

```

```

in init
in init
i5 8gb 50000
i7 16 100000
i5 8gb 50000
i7 16 100000

```

In [28]: `class computer():`

```

    def __init__(self,cpu,ram): # cpu & ram as argument
        self.cpu = cpu #hear self is the object & we need to refer to the object
        self.ram = ram

    def config(self):
        print('config is', self.cpu, self.ram)

com1 = computer('i5', 8)
com2 = computer('i7', 16)

com1.config()
com2.config()

```

```

config is i5 8
config is i7 16

```

In [29]: `class computer():`

```

    def __init__(self,cpu,ram, price): # cpu & ram as argument

```

```
self.cpu = cpu #hear self is the object & we need to refer to the object
self.ram = ram
self.price = price

def config(self):
    print('config is', self.cpu, self.ram, self.price )

com1 = computer('i5', 8, 50000)
com2 = computer('i7', 16, 70000)

com1.config()
com2.config()
```

```
config is i5 8 50000
config is i7 16 70000
```

Constructore & Self

```
class computer:
```

```
c1 = computer()
```

```
print(id(c1))
```

```
In [31]: class computer:
         pass
```

```
c1 = computer()
print(id(c1))
```

```
2873431071712
```

```
In [32]: class computer:
         pass
```

```
c1 = computer()
c2 = computer()
```

```
print(id(c1))  
print(id(c2))
```

2873431072048

2873444602640

```
In [33]: class computer():  
         def __init__(self):  
             self.name = 'santosh'  
             self.age = 23  
  
         c1 = computer()  
         c2 = computer()  
  
         print(c1.name, c1.age)  
         print(c2.name)
```

santosh 23

santosh

```
In [34]: class computer():  
         def __init__(self):  
             self.name = 'santosh'  
             self.age = 23  
  
         c1 = computer()  
         c2 = computer()  
  
         c2.name = 'mit'  
         c1.name = 'rakesh'  
  
         print(c1.name)  
         print(c2.name)
```

rakesh

mit

oops

- 1- class (complete)

- 2- method (completed)
- 3- object (completed) -- ENCAPSULATION
-
- 4- encapsulation
- 5- inheritance
- 6- polymorphism
- 7- abstraction

ENCAPSULATION (OBJECT)

- VARIABLES

instance variable

class variable = static variable

- METHODS

instance method

class method

static method

```
In [1]: class car():
        def __init__(self):
            self.mil = 10
            self.com = 'bmw'

c1 = car()
c2 = car()

print(c1.com, c1.mil)
print(c2.com, c2.mil)

print(id(c1))
print(id(c2))
```

```
bmw 10  
bmw 10  
2775096540672  
2775096573648
```

```
In [2]: class car():  
        def __init__(self):  
            self.mil = 10  
            self.com = 'bmw'  
  
        c1 = car()  
        c2 = car()  
  
        c2.mil = 8  
        c1.mil = 20  
        c1.com = 'Maruti'  
  
        print(c1.com, c1.mil)  
        print(c2.com, c2.mil)
```

```
Maruti 20  
bmw 8
```

```
In [3]: class car():  
        wheels = 5  
  
        def __init__(self):  
            self.mil = 10  
            self.com = 'bmw'  
  
        c1 = car()  
        c2 = car()  
  
        c2.mil = 8  
  
        print(c1.com, c1.mil, c1.wheels)  
        print(c2.com, c2.mil, c2.wheels)
```

```
bmw 10 5  
bmw 8 5
```

```
In [4]: class car():  
  
    wheels = 5  
  
    def __init__(self):  
        self.mil = 10  
        self.com = 'bmw'  
  
c1 = car()  
c2 = car()  
  
c2.mil = 8  
c2.wheels = 4  
  
print(c1.com, c1.mil, c1.wheels)  
print(c2.com, c2.mil, c2.wheels)
```

```
bmw 10 5  
bmw 8 4
```

```
In [5]: class student():  
  
    school = 'nit' #school is class variable  
  
    def __init__(self, m1, m2, m3) : # hear i pass 3 variable (m1,m2,m3 are instance variable)  
        self.m1 = m1  
        self.m2 = m2  
        self.m3 = m3  
  
    #lets create a method to calculate average of the marks  
    def avg(self): # -----> hear this average is INSTANCE METHOD  
        return (self.m1+self.m2+self.m3)/3  
  
    # Lets create object hear as we created s1, s2 we created 2 object & also i want to pass value in this  
s1 = student(10, 20, 30)  
s2 = student(5, 15, 10)  
  
print(s1.avg())  
print(s2.avg())
```

20.0

10.0

In [6]: `class student():` `school = 'nit'` `def __init__(self, m1, m2, m3) : # hear i pass 3 variable` `self.m1 = m1` `self.m2 = m2` `self.m3 = m3``#lets create a method to calculate average of the marks` `def avg(self): # -----> hear this average is INSTANCE METHOD` `return (self.m1+self.m2+self.m3)/3` `def info(cls): # user need to define cls - for class variable` `return cls.school``# Lets create object hear as we created s1, s2 we created 2 object & also i want to pass value in this``s1 = student(10, 20, 30)``s2 = student(5, 15, 10)``print(s1.avg())``print(s2.avg())``print(s1.info())``print(s2.info())`

20.0

10.0

nit

nit

In [7]: `class student():` `school = 'nit'` `def __init__(self, m1, m2, m3) : # hear i pass 3 variable` `self.m1 = m1` `self.m2 = m2` `self.m3 = m3`

```

#lets create a method to calculate average of the marks

def avg(self):  # -----> hear this average is INSTANCE METHOD
    return (self.m1+self.m2+self.m3)/3

@classmethod
def info(cls): # user need to define cls - for class variable
    return cls.school

# Lets create object hear as we created s1, s2 we created 2 object & also i want to pass value in this
s1 = student(10, 20, 30)
s2 = student(5, 15, 10)

student.info()

```

Out[7]: 'nit'

```

In [8]: class student():

    school = 'nit'

    def __init__(self, m1, m2, m3) : # hear i pass 3 variable
        self.m1 = m1
        self.m2 = m2
        self.m3 = m3

    #lets create a method to calculate average of the marks
    def avg(self):  # -----> hear this average is INSTANCE METHOD
        return (self.m1+self.m2+self.m3)/3

    @classmethod
    def getSchool(cls):
        return cls.school

    @staticmethod # this student class nothing to do any class & for that also u need to pass speciasl decorator
    def info():
        print('this is student class')

# Lets create object hear as we created s1, s2 we created 2 object & also i want to pass value in this
s1 = student(10, 20, 30)
s2 = student(5, 15, 10)

```

```
print(s1.avg())
print(s2.avg())

print(student.getSchool())

student.info()
```

```
20.0
10.0
nit
this is student class
```

Inheritance

```
In [9]: class A() :
        def feature1(self):          # Lets take example class A has 2 feature1 & feature 2
            print('feature1 is hi')
        def feature2(self):
            print('feature2 is hi')

        # the way you access the feature is object

a1 = A()

a1.feature1()
a1.feature2()
```

```
feature1 is hi
feature2 is hi
```

```
In [15]: class A():
        def feature1(self):          # Lets take example class A has 2 feature1 & feature 2
            print('feature1 is hi')
        def feature2(self):
            print('feature2 is hi')

        class B(A):
            def feature3(self):
```

```

        print('feature3 is bye')
    def feature4(self):
        print('feature4 is welcome')
a = A()
b = B()

b.feature4

```

Out[15]: <bound method B.feature4 of <__main__.B object at 0x000002861F6ACEC0>>

In [14]:

```

class A():
    def feature1(self):          # Lets take example class A has 2 feature1 & feature 2
        print('feature1 is hi')
    def feature2(self):
        print('feature2 is hi')

class B():
    def feature3(self):
        print('feature3 is bye')
    def feature4(self):
        print('feature4 is welcome')
class C(B):
    def feature5(self):
        print('feature5 is goodbye')

a = A()
b = B()
c = C()

c.feature4

```

Out[14]: <bound method B.feature4 of <__main__.C object at 0x000002861F6AC830>>

In [13]:

```

class A():
    def feature1(self):          # Lets take example class A has 2 feature1 & feature 2
        print('feature1 is hi')
    def feature2(self):
        print('feature2 is hi')

class B():

```

```

def feature3(self):
    print('feature3 is bye')
def feature4(self):
    print('feature4 is welcome')
class C(A, B):
    def feature5(self):
        print('feature5 is goodbye')

a = A()
b = B()
c = C()

c.feature1

```

Out[13]: <bound method A.feature1 of <__main__.C object at 0x000002861F6AC1A0>>

```

In [16]: class A(): #Lets create a class hear it should be employee class or student class & Lets create 2 methods
    def __init__(self):
        print('in A init')
    def feature1(self):          # Lets take example class A has 2 feature1 & feature 2
        print('feature1 is hi')
    def feature2(self):
        print('feature2 is hi')

    class B():
        def feature3(self):      # Lets take example class A has 2 feature1 & feature 2
            print('feature3 is hallo')
        def feature4(self):
            print('feature4 is hallo')

a1 = A() # A is the constructor
b1 = B()

```

in A init

```

In [17]: class A(): #Lets create a class hear it should be employee class or student class & Lets create 2 methods
    def __init__(self):
        print('in A init')
    def feature1(self):          # Lets take example class A has 2 feature1 & feature 2
        print('feature1 is hi')
    def feature2(self):

```

```

        print('feature2 is hi')

class B(A):
    def feature3(self):          # Lets take example class A has 2 feature1 & feature 2
        print('feature3 is hallo')
    def feature4(self):
        print('feature4 is hallo')

a1 = A() # A is the constructor
b1 = B()

```

in A init

in A init

```

In [18]: class A(): #Lets create a class hear it should be employee class or student class & Lets create 2 methods
    def __init__(self):
        print('in A init')
    def feature1(self):          # Lets take example class A has 2 feature1 & feature 2
        print('feature1 is hi')
    def feature2(self):
        print('feature2 is hi')

class B():
    #def __init__(self):
    #    print('in B init')
    def feature3(self):          # Lets take example class A has 2 feature1 & feature 2
        print('feature3 is hallo')
    def feature4(self):
        print('feature4 is hallo')

#a1 = A() # A is the constructor
b1 = B()

```

```

In [19]: class A(): #Lets create a class hear it should be employee class or student class & Lets create 2 methods
    def __init__(self):
        print('in A init')
    def feature1(self):          # Lets take example class A has 2 feature1 & feature 2
        print('feature1 is hi')
    def feature2(self):
        print('feature2 is hi')

```

```

class B(A):
    #def __init__(self):
    #    print('in B init')
    def feature3(self):          # Lets take example class A has 2 feature1 & feature 2
        print('feature3 is hallo')
    def feature4(self):
        print('feature4 is hallo')

#a1 = A() # A is the constructor
b1 = B()

```

in A init

In [20]:

```

class A(): #Lets create a class hear it should be employee class or student class & Lets create 2 methods
    def __init__(self):
        print('in A init')
    def feature1(self):          # Lets take example class A has 2 feature1 & feature 2
        print('feature1 is hi')
    def feature2(self):
        print('feature2 is hi')

class B(A):
    def __init__(self):
        print('in B init')
    def feature3(self):          # Lets take example class A has 2 feature1 & feature 2
        print('feature3 is hallo')
    def feature4(self):
        print('feature4 is hallo')

#a1 = A() # A is the constructor
b1 = B()

```

in B init

In [21]:

```

class A() :
    def __init__(self):
        print('in the A init')
    def feature1(self):          # Lets take example class A has 2 feature1 & feature 2
        print('feature1 is hi')
    def feature2(self):
        print('feature2 is hi')

```

```
class B(A):  
    def __init__(self):  
        super().__init__() #when we pass super() == print parent class information as well  
        print('in the B init')  
    def feature3(self): # Lets take example class A has 2 feature1 & feature 2  
        print('feature3 is hallo')  
    def feature4(self):  
        print('feature4 is hallo')  
  
a1 = B()
```

in the A init

in the B init

In []: